

Instituto Tecnológico de Costa Rica
Lenguajes de Programación

Proyecto Programado 4
Colonias Galácticas

Estudiantes:

Luis Trejos Rivera, carnet 2022437816
Esteban Rodríguez Vargas, carnet 2022131105

Profesor:

Ing. Allan Rodríguez Davila

I Semestre 2026

Tabla de Contenidos

Tabla de Contenidos	2
1. Manual de Usuario	4
1.1 Instrucciones de Compilacion	4
Requisitos previos	4
Instrucciones de instalacion	4
1.2 Ejecucion y Uso	5
Iniciar el backend	5
Iniciar el frontend.....	5
Flujo de uso del juego	5
2. Pruebas de Funcionalidad.....	6
2.1 Pantalla principal y autenticacion.....	6
2.2 Creacion de partida	6
2.3 Sala de espera (lobby).....	7
2.4 Inicio de partida y cuenta regresiva	8
2.5 Mapa galactico.....	9
2.6 Panel de informacion de sistema.....	10
2.7 Construccion de instalaciones	10
2.8 Movilizacion de flotas.....	11
2.9 Resolucion de combate	11
2.10 Finalizacion de partida y resultados.....	12
2.11 Ranking historico	13
3. Descripcion del Problema	14
4. Diseno del Programa.....	14
4.1 Diagrama de Paquetes	14
Paquete Backend (Node.js)	14
Paquete Frontend (React).....	15
4.2 Diagrama de Distribucion	15
4.3 Diagrama de Comunicacion	15
5. Librerias Usadas	16
5.1 Backend.....	16
5.2 Frontend	16
6. Analisis de Resultados	17
6.1 Objetivos Alcanzados	17

6.2 Objetivos No Alcanzados y Razones	18
7. Descripcion Manual de los Principales Algoritmos	18
7.1 Construccion del grafo de adyacencia	18
7.2 Ciclo de produccion de recursos	18
7.3 Resolucion de combate	18
7.4 Calculo de ranking y condiciones de victoria	18
8. Bitacora	19

1. Manual de Usuario

1.1 Instrucciones de Compilación

Requisitos previos

- Sistema operativo Windows 10, Windows 11, macOS o Linux
- Node.js versión 18 o superior
- PostgreSQL instalado localmente, o acceso a una instancia remota de PostgreSQL
- Git para clonar el repositorio
- Conexión a internet para la descarga de dependencias

Instrucciones de instalación

1. Clonar el repositorio desde GitHub:

```
git clone https://github.com/LuisK19/P04-ColoniasGalacticas.git
cd colonias-galacticas
```

2. Instalar las dependencias del backend:

```
cd backend
npm install
```

3. Instalar las dependencias del frontend:

```
cd frontend
npm install
```

4. Configurar la base de datos PostgreSQL:

- Crear una base de datos llamada colonias_galacticas en PostgreSQL local, usando pgAdmin, DBeaver o psql.
- Ejecutar el archivo backend/src/db/schema.sql en esa base de datos para crear las tablas necesarias.

5. Configurar las variables de entorno del backend en un archivo .env dentro de la carpeta backend, con el siguiente contenido de referencia:

```
PORT=3000
DB_MODE=local
DATABASE_URL_LOCAL=postgresql://postgres:TU_PASSWORD@localhost:5432/colonias_galacticas
DATABASE_URL_REMOTA=postgresql://usuario:password@host:5432/colonias_galacticas
JWT_SECRET=colonias_galacticas_secret_2026
CICLO_PRODUCCION_SEG=20
PORCENTAJE_VICTORIA=70
```

```
TIEMPO_ESPERA_LOBBY_SEG=60
```

El parámetro DB_MODE permite alternar entre una base de datos local y una remota sin modificar el código, simplemente cambiando su valor entre local y remota.

6. Configurar la variable de entorno del frontend en un archivo .env dentro de la carpeta frontend:

```
VITE_BACKEND_URL=http://localhost:3000
```

1.2 Ejecución y Uso

Una vez completada la instalación, se deben iniciar ambos servidores en terminales separadas.

Iniciar el backend

```
cd backend  
npm run dev
```

El servidor debe mostrar en la terminal el mensaje Servidor corriendo en <http://localhost:3000> y la confirmación de conexión a la base de datos.

Iniciar el frontend

```
cd frontend  
npm run dev
```

Vite mostrara una dirección local, normalmente <http://localhost:5173>, donde se puede acceder a la aplicación desde el navegador.

Flujo de uso del juego

7. Al abrir la aplicación, el jugador ingresa un nickname en la pantalla principal.
8. El jugador elige entre Crear Partida, Unirse a Partida o Ver Ranking.
9. Al crear una partida se configura el nombre, la galaxia, la cantidad mínima y máxima de jugadores, el tiempo máximo de duración y el nivel de recursos iniciales.
10. Los jugadores ingresan a una sala de espera (lobby) donde se actualiza en tiempo real la lista de jugadores conectados.
11. El jugador que crea la partida actúa como host y es el único que puede iniciarla, ya sea presionando la tecla U o el botón correspondiente, una vez que se alcanza la cantidad mínima de jugadores configurada.
12. Al iniciar, se muestra una cuenta regresiva de tres segundos visible para todos los jugadores.
13. Durante la partida, cada jugador puede hacer clic sobre los sistemas planetarios del mapa para ver su información, construir instalaciones si los controla, y movilizar flotas hacia sistemas conectados.

14. La partida finaliza cuando un jugador controla el porcentaje configurado de sistemas, cuando se alcanza el tiempo máximo, o cuando solo queda un jugador activo.
15. Al finalizar, se muestra la pantalla de resultados con las estadísticas de todos los jugadores, y se puede consultar el ranking histórico de partidas anteriores.

2. Pruebas de Funcionalidad

En esta sección se documenta cada funcionalidad principal del sistema junto con la evidencia visual de su correcto funcionamiento.

2.1 Pantalla principal y autenticación

Al ejecutar el frontend, se presenta la pantalla principal donde el jugador ingresa su nickname antes de poder acceder a cualquier funcionalidad del juego.



2.2 creación de partida

El jugador configura el nombre de la partida, selecciona la galaxia, define la cantidad mínima y máxima de jugadores, el tiempo máximo y el nivel de recursos iniciales.

[< Volver](#)

Nueva Partida

Nombre de la partida

Ej: Batalla por Orion

Galaxia

Nebulosa Orion (25 sistemas, 41 rutas) ▾

Mín. jugadores

2

Máx. jugadores

4

Tiempo máximo (minutos)

30

Recursos iniciales

Bajo

100 / 50 / 20

Normal

300 / 150 / 50

Alto

500 / 250 / 100

 Crear Partida

2.3 Sala de espera (lobby)

Tras crear o unirse a una partida, el jugador ingresa a la sala de espera. La lista de jugadores conectados se actualiza en tiempo real mediante WebSockets.



2.4 Inicio de partida y cuenta regresiva

Al presionar el botón de inicio o la tecla U, el host dispara una cuenta regresiva de tres segundos visible para todos los jugadores antes de comenzar la partida.



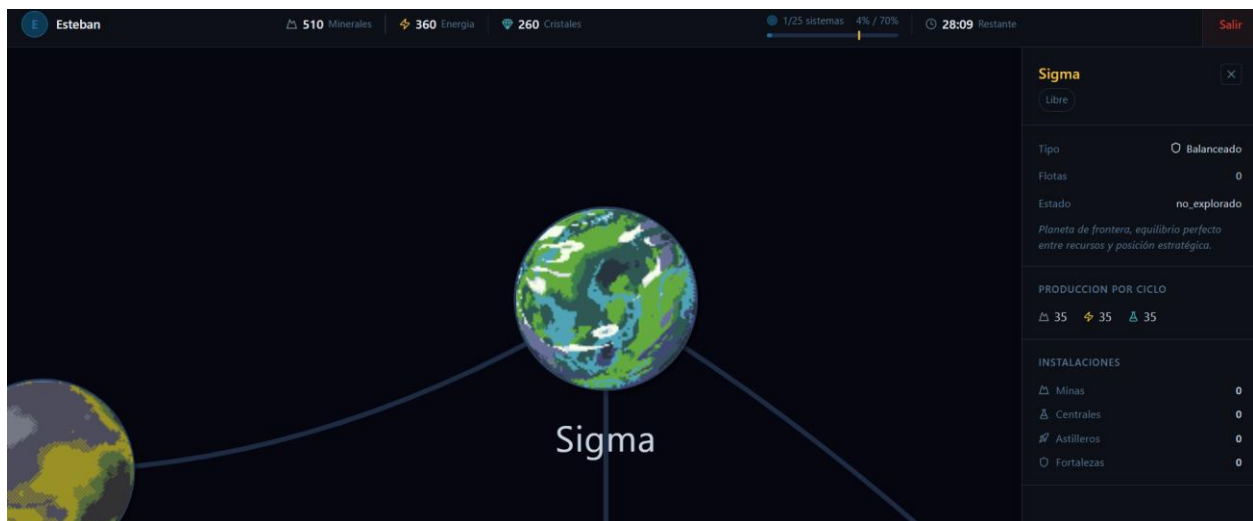
2.5 Mapa galáctico

El mapa muestra el grafo completo de sistemas planetarios, con imágenes representativas según el tipo de planeta y coloreado según el propietario de cada sistema.



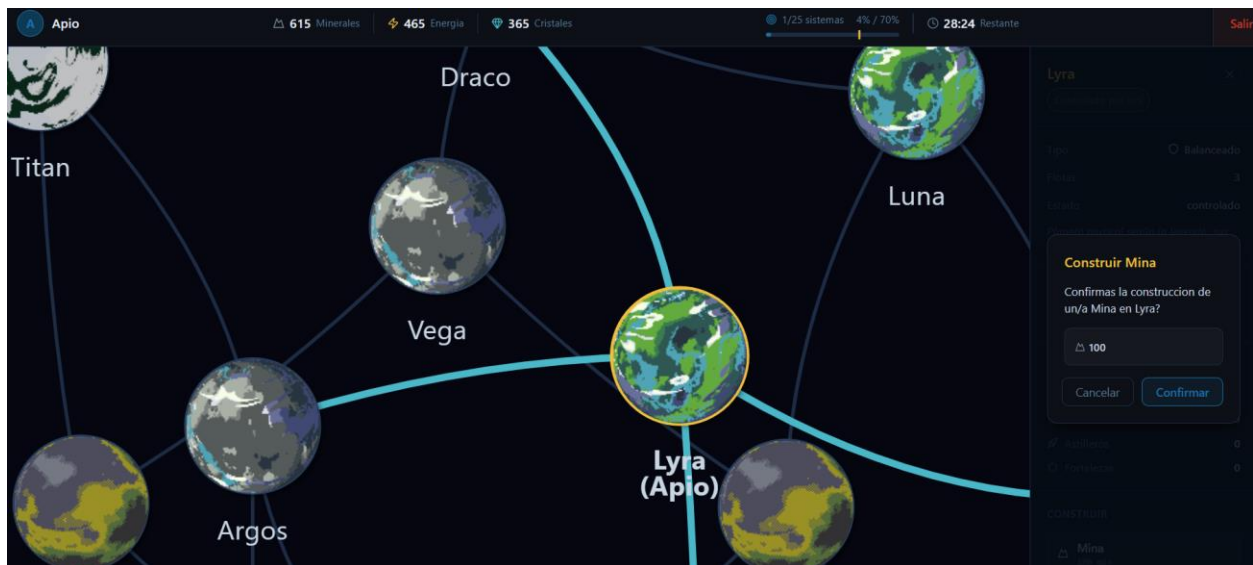
2.6 Panel de información de sistema

Al hacer clic sobre un sistema planetario se despliega un panel lateral con su nombre, propietario, tipo, producción de recursos, instalaciones y opciones de acción.



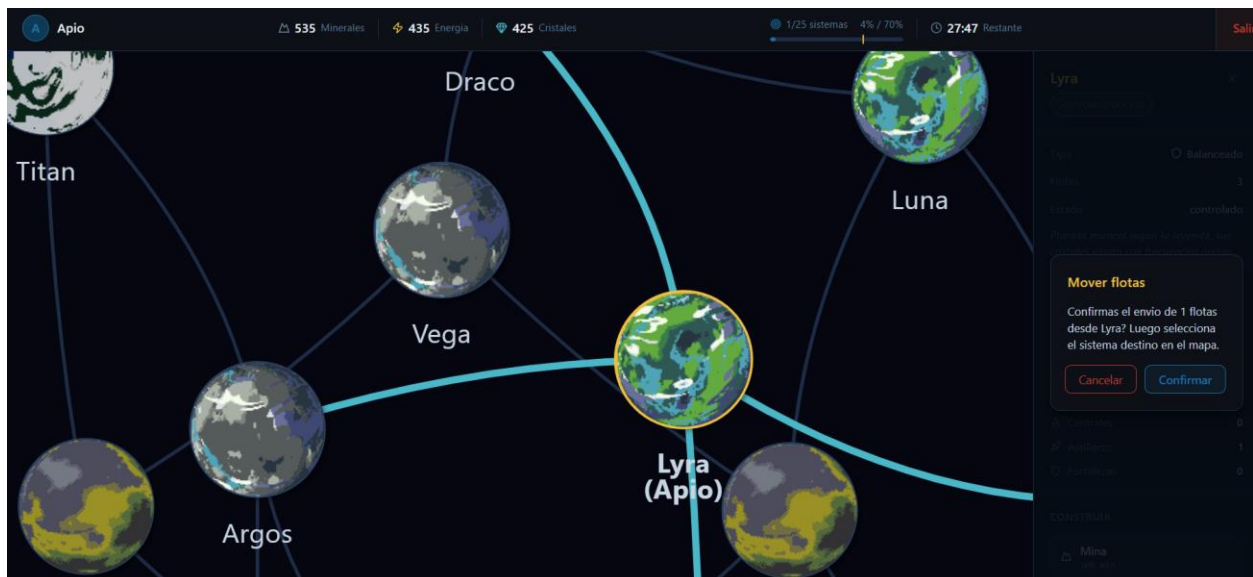
2.7 Construcción de instalaciones

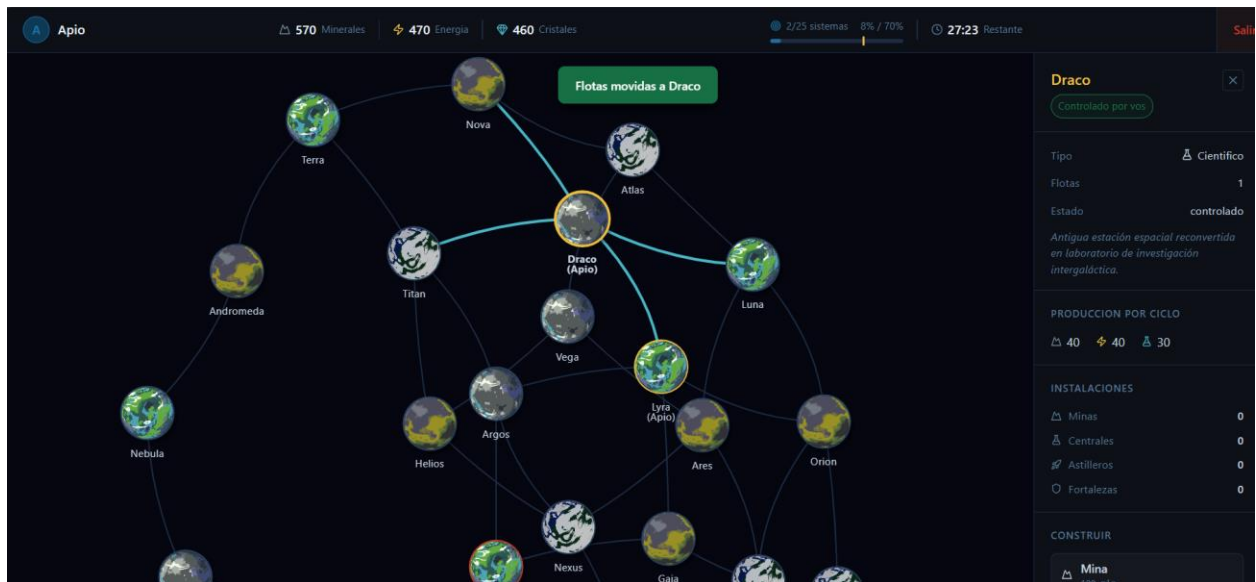
Sobre un sistema propio, el jugador puede construir minas, centrales de investigación, astilleros o fortalezas, siempre que disponga de los recursos necesarios.



2.8 Movilización de flotas

El jugador selecciona la cantidad de flotas a mover desde un sistema propio y elige el sistema destino directamente en el mapa, el cual se resalta visualmente durante la selección.





2.9 Resolución de combate

Cuando un jugador envía flotas hacia un sistema controlado por otro jugador, el sistema resuelve automáticamente el combate aplicando las reglas de neutralización de flotas, minas y fortalezas.



2.10 Finalización de partida y resultados

Al cumplirse alguna condición de victoria, el sistema calcula el puntaje final de cada jugador y los redirige automáticamente a la pantalla de resultados.



Resultados

Ganador: **EIRV**

🌐 Nebulosa Orion
🕒 0:00 de juego
👤 2 jugadores

#	JUGADOR	PUNTAJE	SISTEMAS	MINERALES	ENERGÍA	CRISTALES	FLOTAS	MINAS	CENTROS	FORTALEZAS
1	EIRV	8 310	1	1 600	570	190	3	1	0	0
2	Apio	8 270	1	1 000	850	190	3	0	0	0

[Ver Ranking Global](#)
[Volver al Inicio](#)

2.11 Ranking histórico

La pantalla de ranking muestra el listado de partidas finalizadas, ordenadas por fecha, con el ganador y sus estadísticas principales.

[< Volver](#)


Ranking Global

EIRV

🌐 Nebulosa Orion
🕒 5:00
23/6/2026

1
Sistemas
 1 600
Minerales
 570
Energía
 190
Cristales

ID: e1ab6ed8...

Pablo

🌐 Nebulosa Orion
🕒 5:00
23/6/2026

1
Sistemas
 790
Minerales
 640
Energía
 540
Cristales

ID: e58c3acc...

Pablo

🌐 Nebulosa Orion
🕒 30:00
23/6/2026

1
Sistemas
 9 200
Minerales
 2 820
Energía
 940
Cristales

ID: e7c04458...

4

EIRV

🌐 Nebulosa Orion
🕒 30:00
23/6/2026

1
Sistemas
 3 860
Minerales
 3 710
Energía
 2 720
Cristales

ID: f45dae30...

5

EIRV

🌐 Nebulosa Orion
🕒 32:00
23/6/2026

1
Sistemas
 4 000
Minerales
 3 950
Energía
 2 900
Cristales

ID: 2bc2b26e...

3. Descripción del Problema

Los juegos de estrategia y conquista espacial han sido un género recurrente dentro de la industria de los videojuegos, caracterizados por permitir a los jugadores explorar sistemas estelares, administrar recursos y competir o cooperar dentro de universos compartidos. Sin embargo, implementar este tipo de experiencia de forma correcta exige resolver varios problemas técnicos simultáneos: la representación de un universo como una estructura de grafo cargada desde un archivo externo, la sincronización en tiempo real del estado del juego entre múltiples clientes conectados, la administración concurrente de recursos y combates entre jugadores, y la persistencia selectiva de información relevante una vez finalizada cada partida.

El problema central del proyecto Colonias Galácticas consiste en diseñar y construir una aplicación cliente servidor que permita a varios jugadores conectarse simultáneamente a una galaxia compartida, representada como un grafo de sistemas planetarios y rutas espaciales, y competir por el control de dichos sistemas mediante la administración de recursos, la construcción de instalaciones y la movilización de flotas militares. El sistema debe garantizar que los cambios realizados por cualquier jugador se reflejen en tiempo real para el resto de los participantes, manteniendo la coherencia del estado del juego mientras este se ejecuta completamente en memoria durante la partida activa.

Para resolver este problema se optó por separar la aplicación en dos proyectos independientes: un servidor backend desarrollado en Node.js con Express, encargado de toda la lógica del juego y la sincronización mediante WebSockets, y un cliente frontend desarrollado en React con Vite, encargado exclusivamente de la presentación visual y la captura de las acciones del usuario. Esta separación permite que la lógica del dominio del juego sea independiente de su representación gráfica, facilitando el mantenimiento y la extensibilidad del sistema.

4. Diseño del Programa

4.1 Diagrama de Paquetes

El sistema se organiza en dos paquetes principales completamente independientes entre sí, comunicados únicamente a través de la red.

Paquete Backend (Node.js)

- config: contiene la configuración centralizada del juego, incluyendo los valores de producción de recursos, costos de construcción y parámetros ajustables sin necesidad de modificar el código.
- db: contiene la conexión a la base de datos PostgreSQL y el esquema SQL de las tablas persistentes.
- game: contiene la lógica del dominio del juego, específicamente el cargador de galaxias y el administrador de partidas que mantiene el estado en memoria.

- routes: contiene los endpoints REST para autenticación, gestión de partidas, listado de galaxias y consulta del ranking.
- sockets: contiene la lógica de comunicación en tiempo real mediante Socket.IO.

Paquete Frontend (React)

- Pages: contiene las pantallas principales de la aplicación, entre ellas Home, CrearPartida, Partidas, Lobby, Game, Results y Ranking.
- Components: contiene componentes reutilizables como el mapa galáctico, la barra de recursos y el panel de información de sistema.
- Socket: contiene la instancia compartida del cliente de Socket.IO utilizada en toda la aplicación.

4.2 Diagrama de Distribución

La aplicación se distribuye en dos procesos independientes que pueden ejecutarse en la misma máquina o en máquinas distintas dentro de la misma red. El servidor backend expone una API REST en el puerto 3000 y un canal de WebSockets sobre el mismo puerto. El cliente frontend se sirve mediante Vite en el puerto 5173 durante el desarrollo. Ambos servicios se exponen públicamente mediante herramientas de túnel como Ngrok o el reenvío de puertos integrado en Visual Studio Code, permitiendo que jugadores en distintas redes puedan conectarse a la misma partida. La base de datos PostgreSQL puede ejecutarse localmente en la misma máquina que el backend, sin necesidad de exponerse públicamente, ya que solo el servidor backend establece conexión directa con ella.

4.3 Diagrama de Comunicación

La comunicación entre cliente y servidor combina dos protocolos según la naturaleza de la operación. Las operaciones que no requieren actualización inmediata para otros jugadores, como la autenticación por nickname, la creación de una partida o la consulta del ranking histórico, se realizan mediante peticiones HTTP siguiendo el patrón REST. Las operaciones que ocurren durante una partida activa y que deben reflejarse de inmediato en todos los clientes conectados, como el inicio de partida, la construcción de instalaciones, la movilización de flotas y la resolución de combates, se realizan mediante eventos de WebSockets usando la librería Socket.IO. El servidor mantiene una sala de Socket.IO por cada partida activa, de forma que los eventos emitidos solo llegan a los jugadores que participan en esa partida específica.

5. Librerías Usadas

5.1 Backend

Librería	Propósito en el proyecto
express	Framework web para Node.js, utilizado para definir los endpoints REST de autenticación, partidas, galaxias y ranking.
socket.io	Librería para comunicación en tiempo real mediante WebSockets, utilizada para sincronizar el estado del juego entre todos los jugadores conectados a una partida.
pg	Cliente de PostgreSQL para Node.js, utilizado para ejecutar las consultas SQL de persistencia de partidas, jugadores y ranking.
dotenv	Carga de variables de entorno desde el archivo .env, permitiendo configurar el puerto, la base de datos y los parámetros del juego sin modificar el código fuente.
Cors	Middleware para habilitar peticiones entre distintos orígenes, necesario para que el frontend pueda comunicarse con el backend durante el desarrollo.
uuid	Generación de identificadores únicos universales, utilizados para identificar cada partida creada.
nodemon	Herramienta de desarrollo que reinicia automáticamente el servidor al detectar cambios en el código.

5.2 Frontend

Librería	Propósito en el proyecto
React	Librería principal para la construcción de la interfaz de usuario mediante componentes.
React-router-dom	Manejo de rutas y navegación entre las distintas pantallas de la aplicación.
socket.io-client	Cliente de WebSockets utilizado para conectarse al servidor y recibir actualizaciones del estado del juego en tiempo real.
axios	Cliente HTTP utilizado para realizar las peticiones a los endpoints REST del backend.
vis-network y vis-data	Renderizado del mapa galáctico como un grafo interactivo, permitiendo visualizar los sistemas planetarios y sus conexiones.
lucide-React	Librería de iconos utilizada en toda la interfaz para representar acciones y recursos de forma visual.

vite	Herramienta de construcción y servidor de desarrollo para el proyecto de React.
------	---

6. Análisis de Resultados

6.1 Objetivos Alcanzados

Funcionalidad	Implementado
Carga del universo galáctico desde archivo JSON	Si
Construcción del grafo de sistemas y rutas en memoria	Si
Propiedades completas de cada sistema planetario	Si
Autenticación por nickname	Si
Creación de partidas con configuración personalizada	Si
Cantidad mínima y máxima de jugadores configurable	Si
Listado de partidas disponibles en tiempo real	Si
Sala de espera con actualización en tiempo real	Si
Inicio de partida exclusivo del host	Si
Cuenta regresiva de tres segundos antes de iniciar	Si
Asignación de planeta base a cada jugador	Si
Producción automática de recursos por ciclo	Si
Mapa galáctico interactivo en tiempo real	Si
Construcción de instalaciones con costos diferenciados	Si
Movilización de flotas respetando el grafo de conexiones	Si
Resolución automática de combate	Si
Condiciones de finalización de partida	Si
Cálculo de puntaje final y estadísticas por jugador	Si
Persistencia de estadísticas en base de datos	Si
Ranking histórico de partidas	Si
Publicación mediante herramientas de túnel	Si

Todas las funcionalidades obligatorias descritas en el enunciado del proyecto fueron implementadas y verificadas mediante pruebas manuales con múltiples jugadores conectados simultáneamente.

6.2 Objetivos No Alcanzados y Razones

No se logro realizar los puntos extra por temática adicional por cuestiones de tiempo.

7. Descripción Manual de los Principales Algoritmos

7.1 Construcción del grafo de adyacencia

Al cargar una galaxia, el algoritmo recorre la lista de sistemas declarados en el archivo JSON y construye un mapa donde cada sistema se inicializa con su estado por defecto, incluyendo propietario nulo, estado no explorado y cero flotas. Posteriormente recorre la lista de rutas, que representan aristas no dirigidas entre dos sistemas, y construye una lista de adyacencia agregando cada conexión en ambos sentidos. Esto permite representar el universo como un grafo no dirigido en memoria, sobre el cual se pueden validar fácilmente las conexiones directas entre sistemas al momento de mover flotas.

7.2 Ciclo de producción de recursos

Cada partida activa mantiene un temporizador independiente que se ejecuta cada veinte segundos, valor configurable. En cada ejecución, el algoritmo recorre todos los sistemas del mapa y, para aquellos que se encuentran en estado controlado, suma a los recursos del jugador propietario la producción correspondiente según el tipo de planeta, además de un bono adicional por cada central de investigación construida en ese sistema. Este ciclo se ejecuta de forma totalmente independiente para cada partida, permitiendo que múltiples partidas corran de forma simultánea en el mismo servidor sin interferir entre sí.

7.3 Resolución de combate

Cuando un jugador envía flotas hacia un sistema controlado por otro jugador, se ejecuta el algoritmo de resolución de combate. Primero se calculan las bajas contra las instalaciones defensivas, neutralizando minas a razón de tres por cada flota invasora disponible y fortalezas a razón de dos astilleros por cada fortaleza. Una vez descontadas las flotas utilizadas en neutralizar instalaciones, las flotas restantes del invasor se enfrentan directamente contra las flotas del defensor, neutralizándose mutuamente en proporción uno a uno. Finalmente se compara la fuerza restante de ambos bandos: si el invasor mantiene mayor cantidad de flotas, toma el control del sistema conservando las flotas sobrevivientes y los centros de investigación del derrotado, mientras que, si el defensor resulta con mayor fuerza, simplemente se le descuentan las pérdidas sufridas durante el enfrentamiento.

7.4 Calculo de ranking y condiciones de victoria

La verificación de victoria se ejecuta al final de cada ciclo de producción. El algoritmo primero determina si solo queda un jugador activo, en cuyo caso la partida finaliza inmediatamente. Si hay más de un jugador activo, se calcula para cada uno el porcentaje de sistemas controlados respecto al total de sistemas de la galaxia, finalizando la partida si algún jugador alcanza el porcentaje configurado como condición de dominio. Al

finalizar la partida por cualquier motivo, se calcula el puntaje de cada jugador combinando la cantidad de sistemas controlados, los recursos acumulados ponderados de forma distinta según el tipo de recurso, y la cantidad de fortalezas y centros de investigación activos, ordenando a los jugadores de forma descendente para determinar las posiciones finales.

8. Bitácora

Repositorio: <https://github.com/LuisK19/P04-ColoniasGalacticas>